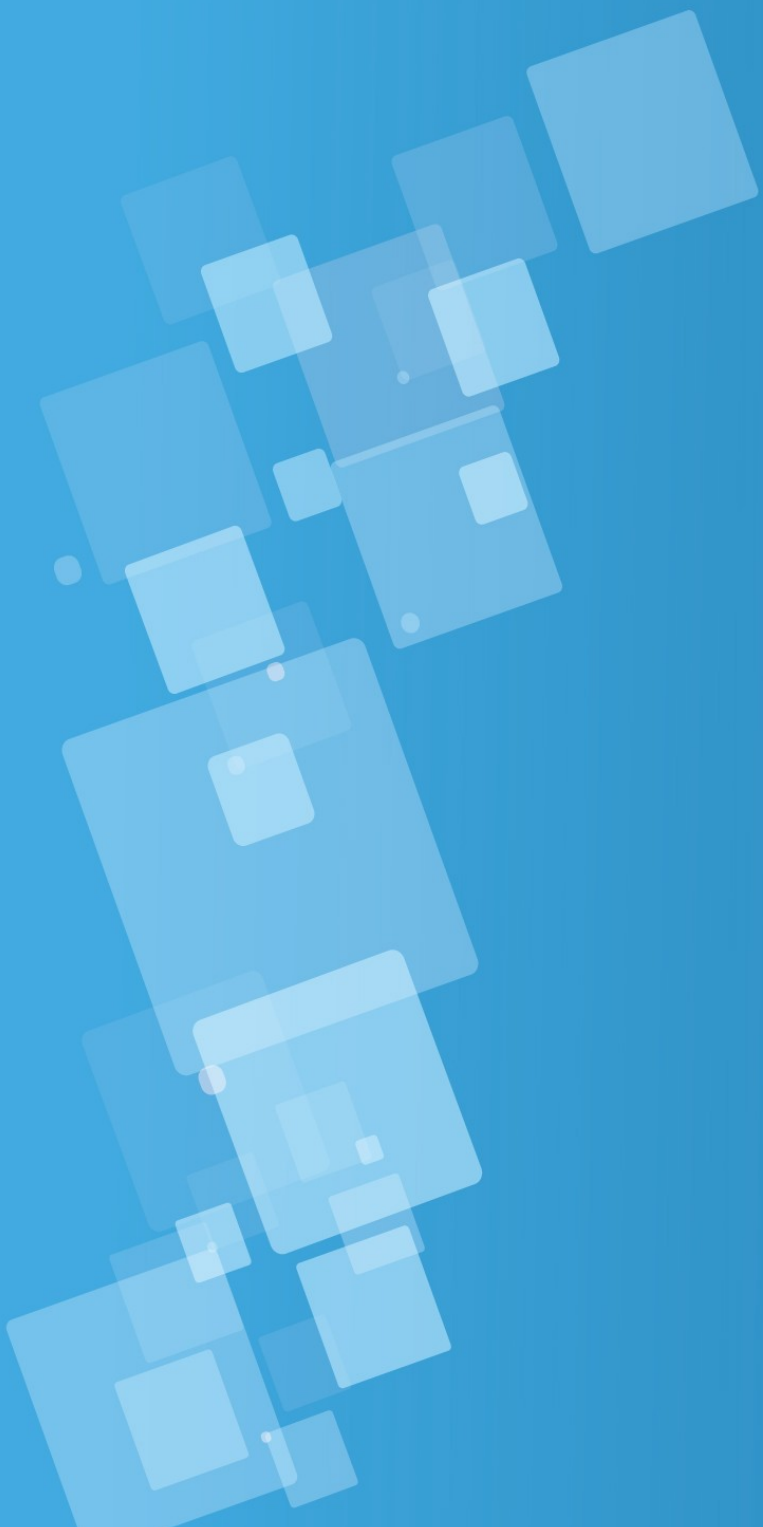




Técnicas Digitales II

Transmisión Serie y SPI (Serial
Peripheral Interface)

- **Bibliografía**

- RM0008.pdf
- SPI Block Guide V03.06 (Motorola)
- Harris & Harris. Digital design and computer architecture: ARM edition. Elsevier, 2015. Capítulo 9.



Interfaz con Periféricos

Control Directo

Cuando los recursos de hardware lo permiten, la conexión entre la CPU y el periférico se realiza de forma dedicada y punto a punto utilizando pines GPIO.

Utilizado en: Dispositivos simples con bajas líneas de control y datos.

Aplicaciones típicas:

- Lectura de teclados matriciales.
- Activación de indicadores LED.
- Control de drivers de motores (Puente H) mediante PWM

Ventajas: Control directo por hardware, máxima velocidad de respuesta y código sencillo.



Interfaz con Periféricos

Interfaz de Datos (Serie o Paralelo)

Cuando la cantidad de GPIO no es suficiente para configurar o leer el dispositivo de manera directa, el mensaje debe ser dividido en partes.

Los pines se agrupan bajo un protocolo para transportar información compleja (bytes/palabras de datos) hacia el mapa de registros del periférico.

Ejemplos: Hablar con una memoria externa, un sensor inteligente o una pantalla.



Transmisión Serie/Paralelo

- Cuando la comunicación con un dispositivo es dividida en partes, podemos realizarla de dos maneras:
 - **Transmisión Serial:** Si en cada paso el mensaje es enviado en una porción de 1 solo bit.
 - **Transmisión Paralela:** Si en cada paso el mensaje es enviado en porciones de 2 o mas bits.



Interfaz Paralela

Históricamente, la forma natural de transferir datos entre integrados fue replicar el bus interno de la CPU hacia el exterior.

- Se transmite una palabra digital completa (8, 16 o 32 bits) de manera simultánea, utilizando un cable físico para cada bit, más líneas de control (Read, Write, Enable).
- **Ventaja:** Máxima tasa de transferencia (Ancho de Banda) debido a la simultaneidad.
- **Desventaja:** Consumo de pines GPIO. Un microcontrolador como la Blue Pill se queda sin pines conectando un solo periférico paralelo complejo.

Memorias RAM/Flash en paralelo, pantallas LCD de alta velocidad.



Interfaz Serie

Para solucionar el costo físico y la falta de espacio en los microcontroladores, la palabra de datos se descompone a su mínima división 1 bit.

- El mensaje paralelo de la CPU se divide en partes y se transmite secuencial mente, bit a bit, a través de una única línea de datos física.
- Un Registro de Desplazamiento (Shift Register) realiza la conversión Paralelo → Serie y Serie → Paralelo
- **Ventaja:** ahorro drástico de pines del chip y reducción de pistas en el PCB.
- **Desventaja:** Se pierde la simplicidad del hardware.



Transmisión Serie

- Por ser la comunicación Serie el principal modo de comunicación con dispositivos, la mayoría de los microcontroladores ofrecen distintos estándares con el hardware necesario para implementarlo de manera sencilla.
- Estándar de comunicación populares son por ejemplo:
 - Inter-Integrated Circuit (I²C)
 - Ethernet
 - Universal Serial Bus (USB)
 - Serial Peripheral Interface (SPI)
 - Universal Asynchronous Receiver/Transmitter (UART)
 - CAN

Transmisión Serie

- 5 de los estándar indicados están disponible en la BluePill, a excepción de Ethernet solo disponible en *Connectivity Line*

- Inter-Integrated Circuit (I²C)

- Ethernet

- Universal Serial Bus (USB)

- Serial Peripheral Interface (SPI)

- Universal Asynchronous Receiver/Transmitter (UART)

- CAN

Veremos una breve descripción del protocolo

En particular veremos SPI y UART, sus característica e implementación en la Blue Pill



Serial Peripheral Interface (SPI)



Descripción General

- Protocolo de comunicación serie, sencillo de usar y rápido.
- Es síncrono, es decir, receptor y transmisor comparten un reloj, permitiendo enviar y recibir los bits sincronizados a este mismo reloj.
- Fue desarrollado por Motorola a finales de 1980, y se convirtió en un estándar utilizado por muchas empresas.
- Su principal aplicación es en la comunicación de dispositivos dentro de la placa.



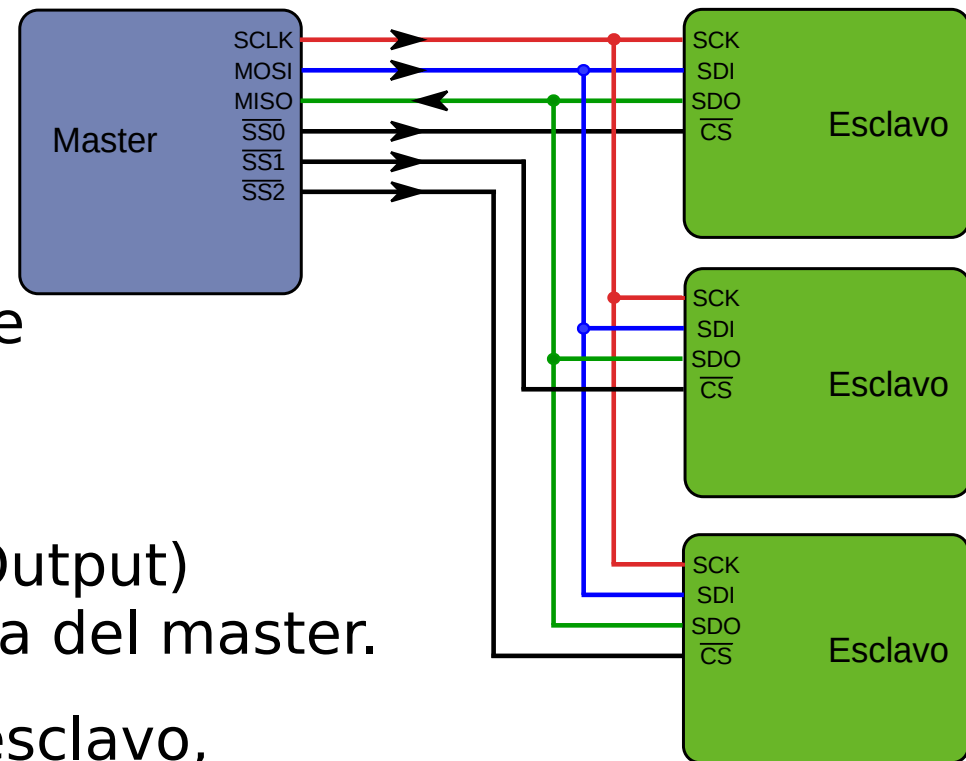
Descripción General

- Protocolo maestro / esclavo generalmente de simple maestro.
- El bus es full-duplex de cuatro líneas de señales y una masa.
- La interfaz física, está compuesta por 3 señales
 - *Serial Clock* (SCK)
 - *Master Out Slave Input* (MOSI también denominado SDO)
 - *Master In Slave Output* (MISO también denominado SDI)
- Permite además múltiple esclavos mediante la selección individual con una 4ta línea $\overline{SS}$ (Slave select) o NSS en ST.

Esquema de la Red

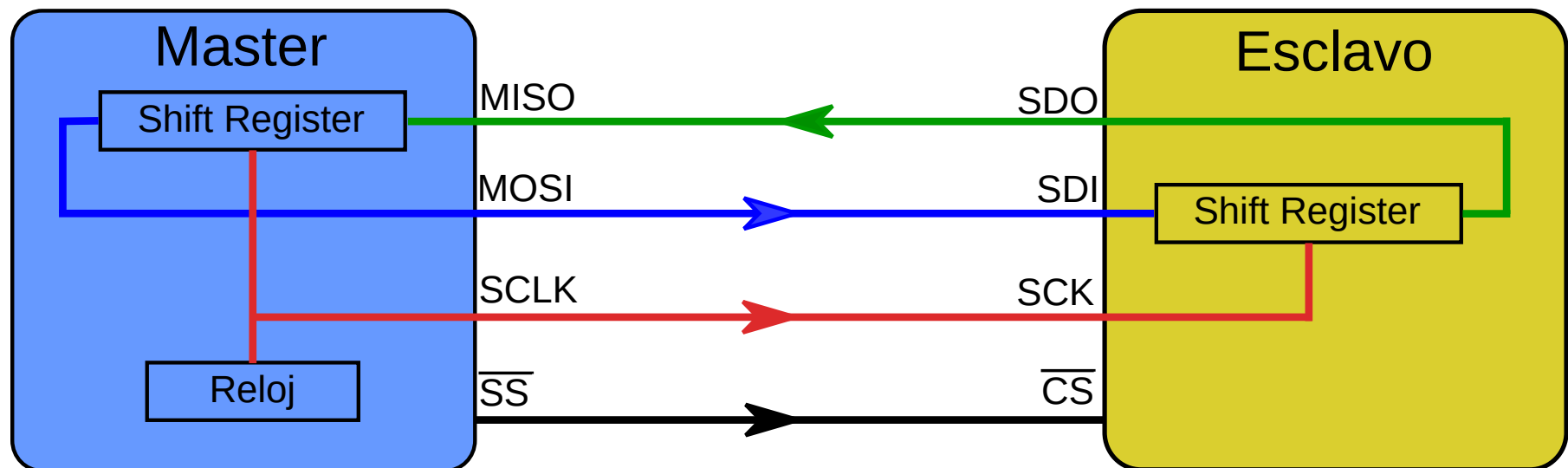
Descripción de las 4 Lineas que permiten la comunicación

- **SCLK** o **SCK** (Serial Clock) Reloj de sincronismo entregado por el master, para el esclavo es la entrada del clock de sincronismo.
- **MOSI** o **SDO** (Master Output Slave Input) Salida de dato del master, entrada del esclavo.
- **MISO** o **SDI** (Master Input Slave Output) Salida de dato del esclavo, entrada del master.
- **$\overline{SS}$** (Slave Select) Habilitador del esclavo, una línea por cada esclavo, permite seleccionar un esclavo a la vez, anulando los demás.



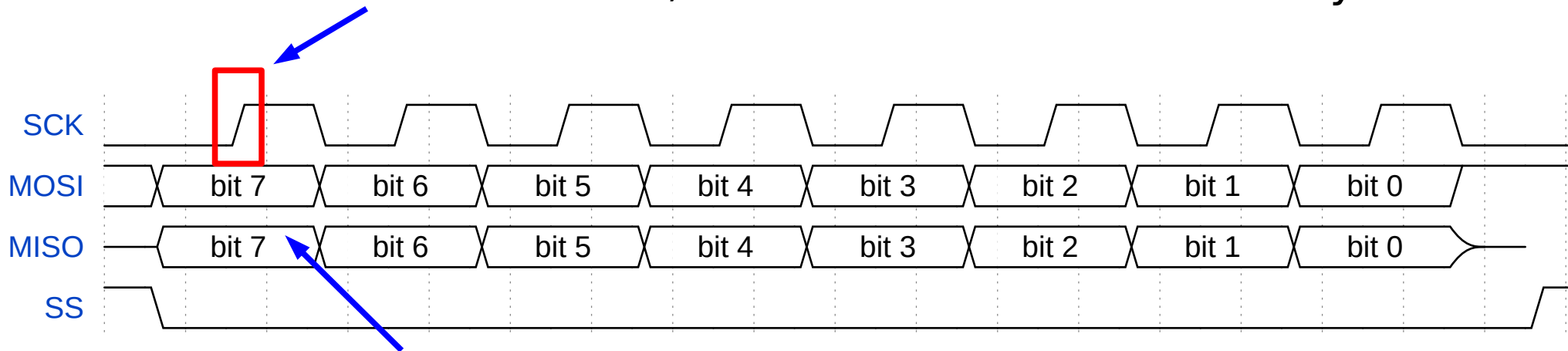
Transmisión de datos

- El master selecciona el esclavo con un 0 en $\overline{SS}$.
- Si el esclavo seleccionado requiere un tiempo de espera, el master debe esperar al menos ese tiempo.
- El Master genera el clock, esto inicializa la comunicación.
- Durante cada ciclo el master envía un bit por **MOSI** y el esclavo lo lee, en ese momento el esclavo envía un bit por **MISO** para que el master lo lea.

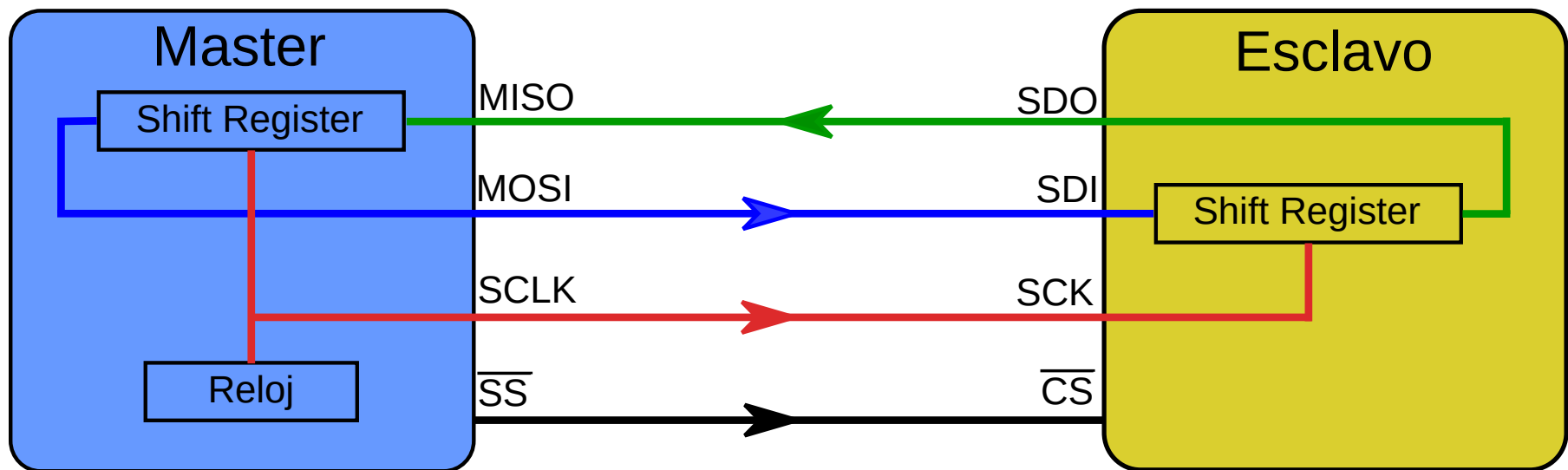


Transmisión de datos

Flanco de subida, se leen los datos en Master y Esclavo



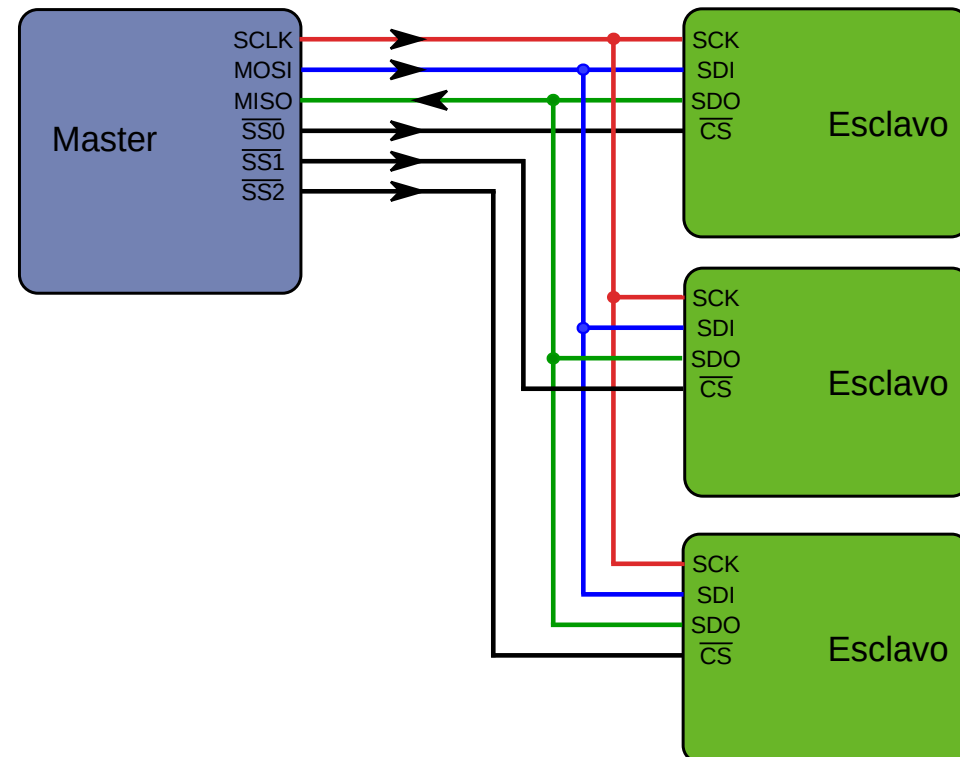
Orden de bits no especificado



Configuración de Esclavos

Esclavos Independientes

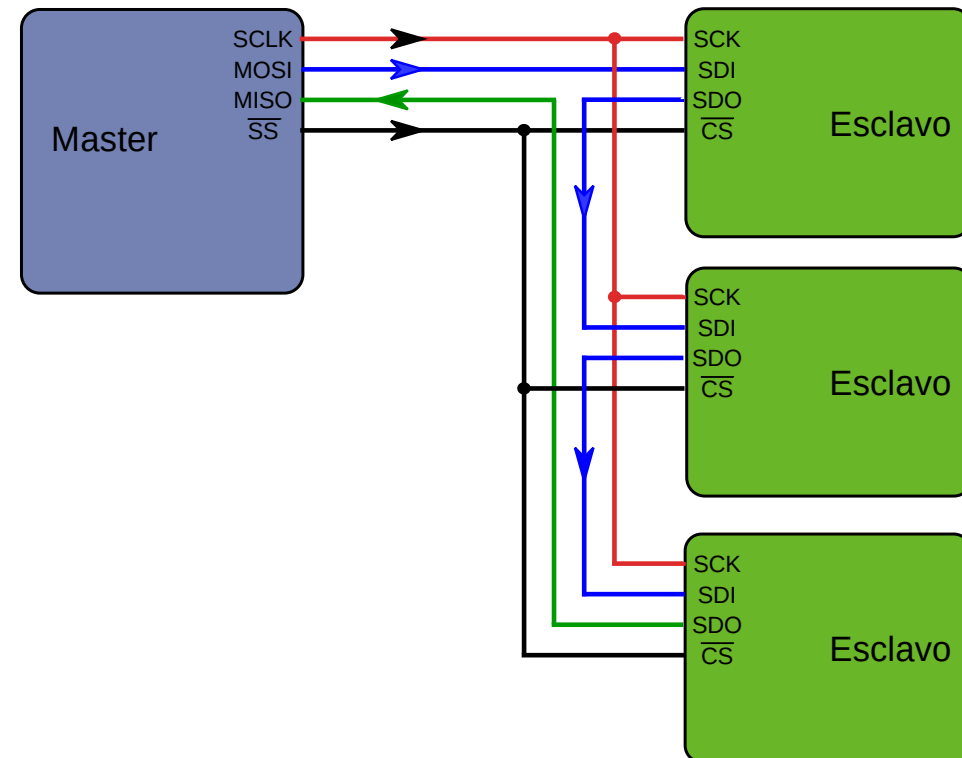
- Cada esclavo tiene su selector.
- Es la configuración mas usada.
- Requiere que los esclavos no carguen la linea cuando están deshabilitados.



Configuración de Esclavos

Cadena (daisy chain)

- Una sola línea va atravesando cada esclavo.
- Un bit ingresa a un esclavo y la salida de su shift, es enviada al próximo.
- Requiere menos cables.
- Utilizado donde no es necesario recibir datos.



Polaridad de Reloj y Fase

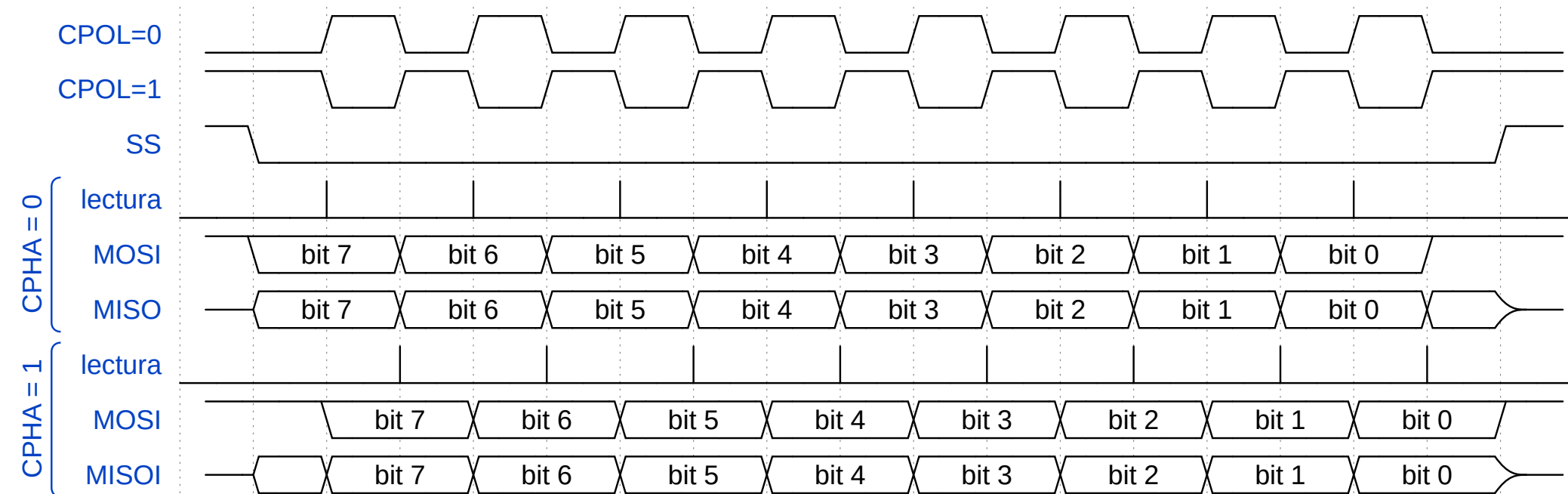
- Según donde se realice la lectura podemos encontrar 4 maneras diferentes de configurar el reloj en el SPI.

Por Polaridad

- CPOL=0 reloj inactivo en 0.
- CPOL=1 reloj inactivo en 1.

Por Fase

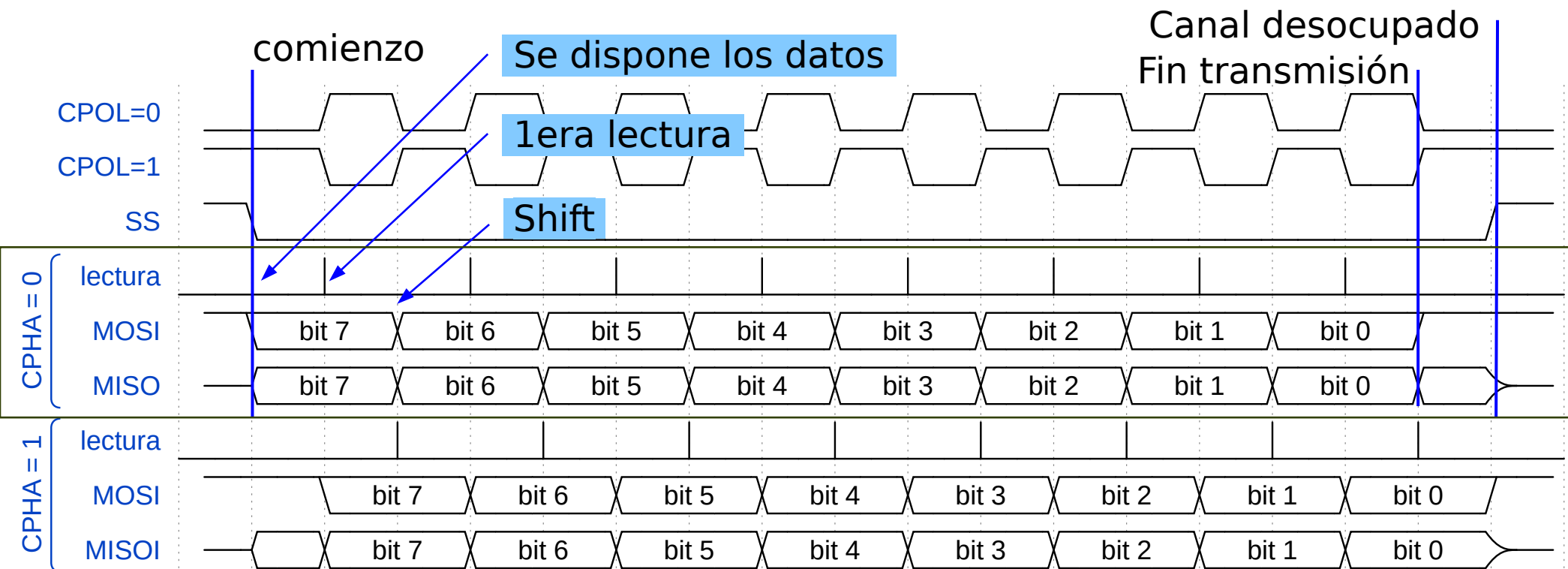
- CPHA=0 utiliza el 1^{er} semiciclo.
- CPHA=1 utiliza el 2^{do} semiciclo.



Polaridad de Reloj y Fase

CPHA = 0 utiliza el 1^{er} semiciclo

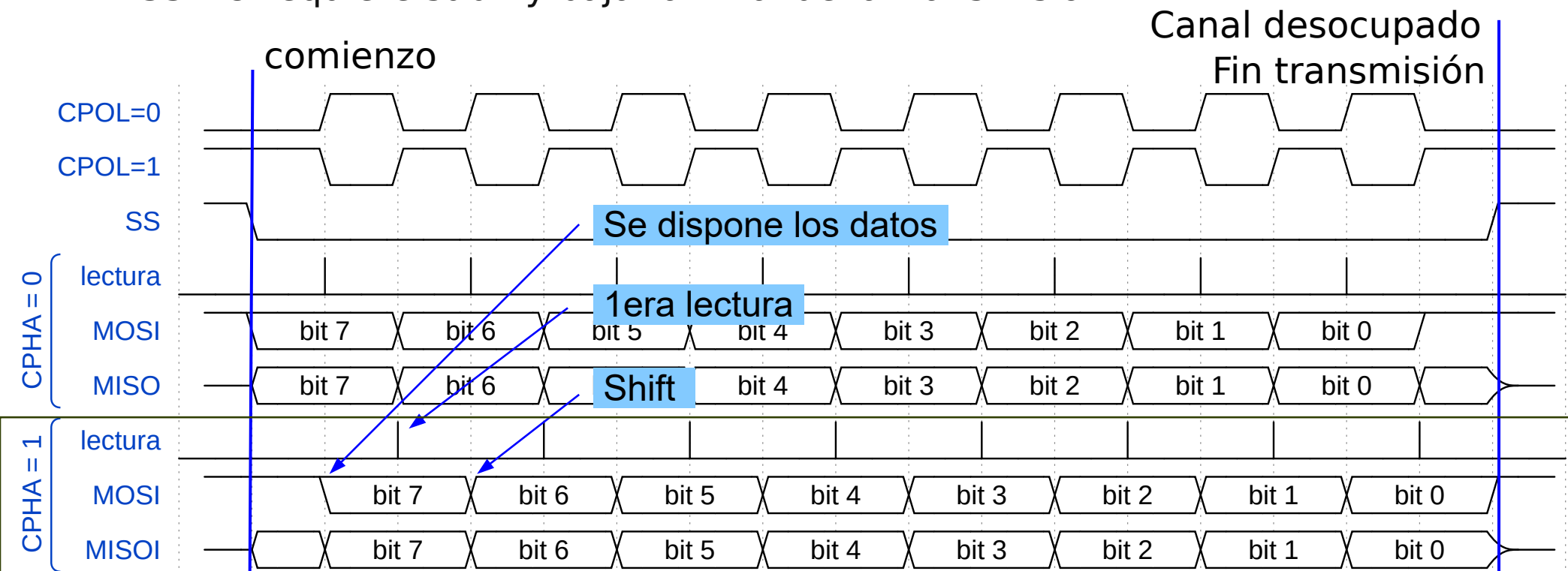
- El 1er bit se dispone al bajar SS.
- En el 1er flanco del clock se realiza la lectura.
- En el 2do flanco del clock se realiza el corrimiento.
- Al final SS requiere subir y luego bajar para enviar otro dato.



Polaridad de Reloj y Fase

CPHA = 1 utiliza el 2^{do} semiciclo

- El 1er bit se dispone en el 1er flanco de clock (requerimiento de algunos dispositivos).
- En el 2do flanco del clock se realiza la lectura.
- En el 3er flanco se realiza el corrimiento.
- SS NO requiere subir y bajar al final de la transmisión.





Ventajas

- Full duplex.
- Push-pull driver, a diferencia del open drain de algunos protocolos seriales, permitiendo trabajar a mayor velocidad.
- El protocolo no esta limitado a una cantidad fija de bits.
- No requiere interfaz ni elementos externo como resistencias.
- Pocas lineas comparada con el paralelo por ejemplo.
- Señales unidireccionales (permite aislamiento galvánico).
- No limitado por el reloj.
- Implementación simple de software.



Desventajas

- Mayor requerimiento de pines que otros protocolos seriales.
- Requiere señales dedicadas para seleccionar un dispositivo.
- Sin control de flujo por parte del esclavo.
- Sin reconocimiento del esclavo, ni chequeo de errores (el master no puede conocer el resultado de una transmisión).
- Típicamente un master.
- No hay estándar formal, existiendo múltiples variaciones.
- Interrupciones deben ser implementadas de manera externa.
- No permite el agregado dinámico de nodos.



SPI en la BluePill



SPI en Blue Pill

- El periférico ST se describe como un módulo SPI/I2S
 - En Blue Pill no se implementa I2S (Inter-IC Sound) protocolo de transmisión digital de audio.
- Permite realizar comunicación Full-Duplex, Half-Duplex, Simplex o línea bidireccional de datos
- Puede seleccionar trama de 8 o 16 bits
- Cambiar la orden de los bits
- Maestro o Esclavo, con entrada para Multimaestro
- Implementa una detección de CRC por hardware
- Gestión NSS por hardware o software



SPI en Blue Pill

Estos dispositivos disponen de 4 pines físicos para realizar la comunicación

- **SCK (Serial Clock):** Salida de reloj si es Maestro y la entrada si es Esclavo. Sincroniza el desplazamiento de los bits.
- **MOSI (Master Out Slave In):** En modo Maestro es una salida de datos; en modo Esclavo es una entrada.
- **MISO (Master In Slave Out):** En modo Maestro es una entrada de datos; en modo Esclavo es una salida.
- **NSS (Slave Select):** Entrada opcional para seleccionar al esclavo. Puede gestionarse por hardware o por software (usando cualquier pin GPIO).



SPI en la Blue Pill

Se implementa 3 módulos SPI independientes

- **SPI1** es la SPI mas rápida, conectada al cabal APB2
- **SPI2/I2S2 y SPI3/I2S3** Estos dispositivos están conectados al APB1, e implementan el I2S para los micros de alta densidad.

Estas característica son de la familia STM32F10{1,2,3,5,7}xx que vemos en el documento principal RM0008.

La BluePill tiene el microcontrolador STM32F103C8T6, este microcontrolador tiene implementado las 2 SPI la SPI1 y SPI2



Registros

- Registro de Estado: SPI→SR (Offset: 0x08) Indica de estado de la SPI
- Registro de Datos: SPI→DR (Offset: 0x0C) Contiene el valor transmitido o recibido.
- Registro Polinomio: SPI→CRCPR (Offset 0x10) Contiene el polinomio matemático usado para el cálculo del CRC.
- SPI→RXCRCR / SPI→TXCRCR Offset (0x14 y 0x18) Solo lectura que contienen el valor CRC calculado de los datos recibidos y transmitidos.
- Registros de Control: SPI→CR1 y SPI→CR2
 - SPI→CR1 (Offset: 0x00): Registro principal de configuración
 - SPI→CR2 (Offset: 0x04): Interrupciones, DMA y configuraciones avanzadas de hardware

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x00	SPI_CR1	Reserved																BIDIMODE	BIDIOE	CRCEN	CRCNEXT	DFE	RXONLY	SSM	SSI	LSBFIRST	SPE	BR [2:0]			MSTR	CPOL	CPHA		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x04	SPI_CR2	Reserved																						TXEIE			RXNEIE	ERRIE	Reserved		SSOE	TXDMAEN	RXDMAEN		
	Reset value																							0	0	0	0		0	0	0				
0x08	SPI_SR	Reserved																						BSY	OVR	MODF	CRCERR	UDR	CHSIDE	TXE	RXNE				
	Reset value																							0	0	0	0	0	0	1	0				
0x0C	SPI_DR	Reserved																DR[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x10	SPI_CRCPR	Reserved																CRCPOLY[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	1	1	1				
0x14	SPI_RXCRCR	Reserved																RxCRC[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x18	SPI_TXCRCR	Reserved																TxCRC[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0				



Configuración del SPI – Selección de Esclavo

- El bit SSM del SPI→CR1 permite gestionar la selección por software o hardware
- Gestión por Software SSM = 1
 - El SPI utiliza el valor del SSI del SPI→CR1 para ver si el esclavo está activo o no
- Gestión por Hardware SSM = 0
 - Aquí opera el pin de hardware NSS
 - Este pin se puede configurar como entrada o salida según el bit SSOE del SPI→CR2

Es habitual que si en modo master se necesita manejar varios esclavos, se opte por el modo software y se maneje los esclavos con GPIO comunes



Configuración del SPI – Selección de Esclavo

- En la gestión de hardware del NSS tenemos entonces dos opciones
- NSS como Salida ($SSM = 0$, $SSOE = 1$)
 - Únicamente en modo maestro. La señal NSS se activa a nivel bajo cuando el maestro inicia la comunicación y permanece en ese nivel hasta que se desactiva el SPI.
- NSS como Entrada ($SSM = 0$, $SSOE = 0$)
 - Como maestro, debe estar en 1, si se lee 0, se interpreta que otro maestro quiere acceder al bus
 - Permite multimaster.
 - Como esclavos, se selecciona cuando NSS está en nivel bajo y se desactiva cuando NSS está en nivel alto.

Configuración del SPI – Selección de Esclavo

Resumen

- SSM Selección por software (SSM=1) o hardware (SSM=0)
- SSI reemplaza al bit NSS cuando SSM=1
- SSOE Configura el pin NSS como salida (SSOE= 1) o entrada (SSOE= 0)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	SPI_CR1	Reserved																BIDIMODE	BIDIOE	CRCEN	CRCNEXT	DFE	RXONLY	SSM	SSI	LSBFIRST	SPE	BR [2:0]			MSTR	CPOL	CPHA	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	SPI_CR2	Reserved																									TXEIE	RXNEIE	ERRIE	Reserved		SSOE	TXDMAEN	RXDMAEN
	Reset value																										0	0	0			0	0	0

Fase y Polaridad del Reloj

Configuración en el registro SPI→CR1

- **CPOL:** define la polaridad
- **CPHA:** define la fase
- **DFF:** define el ancho de la trama, 8 bit para DFF=0 y 16 bit para DFF=1
- **LSBFIRST:** define si el primer bit a enviar es el MSB para LSBFIRST=0 o LSB para LSBFIRST=1

[illegible]

Velocidad de transmisión

Configuración en el registro SPI→CR1

Con los bit BR se configura el divisor de frecuencia

- Para SPI1 $f_{PCLK} = 72\text{MHz}$ máximo
 - Velocidad maxima 36MHz
- Para SPI2 $f_{PCLK} = 36\text{MHz}$ máximo
 - Velocidad máxima 18 MHz

BR[2:0]	Baud rate
000	$f_{PCLK}/2$
001	$f_{PCLK}/4$
010	$f_{PCLK}/8$
011	$f_{PCLK}/16$
100	$f_{PCLK}/32$
101	$f_{PCLK}/64$
110	$f_{PCLK}/128$
111	$f_{PCLK}/256$

[illegible]



Configuración del SPI en modo esclavo

- La configuración del velocidad no es necesaria
- Formato de dato, Polaridad y Fase, Orden de bit, debe configurarse correctamente (según el maestro)
- Gestión de NSS
 - Por Hardware se debe poner el pin NSS en bajo para activar la comunicación
 - Por Software el bit SSM se debe poner en 0
- Modo esclavo MSRT= 0 del SPI→CR1
- Activar con SPE= 1 del SPI→CR1



Configuración del SPI en modo maestro

- Velocidad de Reloj: Seleccionar los bits BR[2:0] en el registro SPI→CR1 para definir el baud rate del reloj serie.
- Formato de dato, Polaridad y Fase, Orden de bit, debe configurarse correctamente
- Gestión de NSS
 - Modo Software: SSM=1 y SSI=1 en SPI→CR1
 - Modo Hardware (Entrada): El pin NSS debe estar a nivel alto durante toda la secuencia de transmisión.
 - Modo Hardware (Salida): SSOE=1 en SPI→CR2
- Modo master MSRT= 1 del SPI→CR1
- Activar con SPE= 1 del SPI→CR1

Banderas

Banderas en el registro SPI→SR

- **TXE** (Tx buffer empty) Indica que el buffer de transmisión está vacío y listo para recibir el siguiente dato a ser enviado. Se borra al escribir en SPI→DR.
- **RXNE** (Rx buffer not empty) Indica que hay un dato válido recibido. Se borra al leer el SPI→DR.
- **BSY** (Busy flag) Indica que se está realizando una comunicación

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x08	SPI_SR Reset value	Reserved																								BSY	OVR	MODF	CRCERR	UDR	CHSIDE	TXE	RXNE
																										0	0	0	0	0	0	1	0

Ejemplo: configurar SPI1 (maestro, FD, 16bits, NSS por software)

1. Debo buscar la salidas de SPI1 (los pines físicos) en el pin out de STM32F103X6.pdf

STM32F103xx

Pin descriptions

Table 3. Pin definitions (continued)

Pins				Pin name	Type ⁽¹⁾	I / O Level ⁽²⁾	Main function ⁽³⁾ (after reset)	Default alternate functions
BGA100	LQFP48	LQFP64	LQFP100					
G3	14	20	29	PA4/SPI1_NSS/ USART2_CK/ADC_IN4	I/O		PA4	SPI1_NSS ⁽⁶⁾ / USART2_CK ⁽⁶⁾ / ADC_IN4
H3	15	21	30	PA5/SPI1_SCK/ ADC_IN5	I/O		PA5	SPI1_SCK ⁽⁶⁾ / ADC_IN5
J3	16	22	31	PA6/SPI1_MISO/ ADC_IN6/TIM3_CH1	I/O		PA6	SPI1_MISO ⁽⁶⁾ / ADC_IN6/TIM3_CH1 ⁽⁶⁾
K3	17	23	32	PA7/SPI1_MOSI/ ADC_IN7/TIM3_CH2	I/O		PA7	
G4	-	24	33	PC4/ADC_IN14	I/O		PC4	

pin 15 → PA5 → SCK
pin 16 → PA6 → MISO
pin 17 → PA7 → MOSI

Ejemplo: configurar SPI1 (maestro, FD, 16bits, NSS por software)

2. Tipo de configuración que debe tener cada pin (tabla 22 a 33 RM0008)

Table 25. SPI

SPI pinout	Configuration	GPIO configuration
SPIx_SCK	Master	Alternate function push-pull
	Slave	Input floating
SPIx_MOSI	Full duplex / master	Alternate function push-pull
	Full duplex / slave	Input floating / Input pull-up
	Simplex bidirectional data wire / master	Alternate function push-pull
	Simplex bidirectional data wire/ slave	Not used. Can be used as a GPIO
SPIx_MISO	Full duplex / master	Input floating / Input pull-up
	Full duplex / slave (point to point)	Alternate function push-pull
	Full duplex / slave (multi-slave)	Alternate function open drain
	Simplex bidirectional data wire / master	Not used. Can be used as a GPIO
	Simplex bidirectional data wire/ slave (point to point)	Alternate function push
	Simplex bidirectional data wire/ slave (multi-slave)	Alternate function open
SPIx_NSS	Hardware master /slave	Input floating/ Input pu
	Hardware master/ NSS output enabled	Alternate function push-pull

pin 15 → PA5 → SCK → AF PP
pin 16 → PA6 → MISO → AF PP
pin 17 → PA7 → MOSI → IF

Ejemplo: configurar SPI1 (maestro, FD, 16bits, NSS por software)

3. Activo Relojes de SPI y del puerto A (ambos del APB2)

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_SPI1EN;
```

4. Configuro los pines como *Alternate Function* o *Input* según corresponda.

```
// Pines PA5(SCK) y PA7(MOSI) como AF Push-Pull (0xB)  
// PA6(MISO) como Input Floating (0x4)  
GPIOA->CRL &= ~(0xFFFF00000); // Limpiar PA5, PA6, PA7  
GPIOA->CRL |= (0xB << 20); // PA5: AFPP 50MHz  
GPIOA->CRL |= (0x4 << 24); // PA6: Input Floating  
GPIOA->CRL |= (0xB << 28); // PA7: AFPP 50MHz
```

Ejemplo: configurar SPI1 (maestro, FD, 16bits, NSS por software)

5. Configurar registro CR1

- En este punto se configura según el detalle
- SSI=1 para que el master funcione, si SSI=0 se considera que otro master quiere entrar y tendremos un error.
- Finalmente se habilita el periférico

```
SPI1->CR1 = 0; // Reset
SPI1->CR1 |= SPI_CR1_DFF; // 16-bit data frame
SPI1->CR1 |= SPI_CR1_MSTR; // Modo Master
SPI1->CR1 |= (0x3 << 3); // Bits[5:3] Baud rate=f/16
SPI1->CR1 |= (0 << 1) | (0 << 0); // CPOL y CPHA en 0
SPI1->CR1 |= SPI_CR1_SSM; // Gestión de NSS por software
SPI1->CR1 |= SPI_CR1_SSI; // SSI=1
SPI1->CR1 |= SPI_CR1_SPE; // Habilitar SPI
```


Ejemplo: configurar SPI1 (maestro, FD, 16bits, NSS por software)

6. Función de Envío y Recepción: La comunicación SPI es Full-Duplex, la forma más eficiente y estándar de programar esto es mediante una función de envío y recepción.

```
uint16_t SPI1_TransmitReceive16(uint16_t data) {  
    // Esperar hasta que la transmisión esté vacía (TXE=1)  
    while (!(SPI1->SR & SPI_SR_TXE));  
    // Escribir el dato en el registro SPI_DR  
    SPI1->DR = data;  
    // Esperar hasta la recepción tenga el dato (RXNE = 1)  
    while (!(SPI1->SR & SPI_SR_RXNE));  
    // La lectura de DR limpia la bandera RXNE.  
    return (uint16_t)SPI1->DR;  
}
```



¿ Preguntas ?